

Here's a minimal, self-contained example: a REST endpoint that uses a **Redisson distributed lock** and a **distributed atomic counter** — the two structures that best show off what "distributed" actually buys you over a plain in-memory `HashMap` or `AtomicLong`.

1. Dependency

```
<dependency>
  <groupId>org.redisson</groupId>
  <artifactId>redisson-spring-boot-starter</artifactId>
  <version>3.34.1</version>
</dependency>
```

2. `application.properties`

```
spring.data.redis.host=localhost
spring.data.redis.port=6379
```

(Redisson's Spring Boot starter auto-detects these and configures a `RedissonClient` bean for you — no extra config class needed.)

3. The demo

```
@RestController
@RequestMapping("/counter")
public class CounterController {

    private final RedissonClient redisson;

    public CounterController(RedissonClient redisson) {
        this.redisson = redisson;
    }

    @PostMapping("/increment")
    public String increment() throws InterruptedException {
        RLock lock = redisson.getLock("counter-lock");
        RAtomicLong counter = redisson.getAtomicLong("shared-counter");

        boolean acquired = lock.tryLock(5, 10, TimeUnit.SECONDS); // wait up to 5s,
        // auto-release after 10s
        if (!acquired) {
            return "Could not acquire lock — another instance is updating the counter";
        }

        try {
```

```

        long value = counter.incrementAndGet(); // atomic, shared across ALL app instances
        return "Counter is now: " + value;
    } finally {
        lock.unlock();
    }
}

@GetMapping("/value")
public long getValue() {
    return redisson.getAtomicLong("shared-counter").get();
}
}

```

Why this demonstrates "distributed" specifically

Run **two instances** of this app on different ports (`--server.port=8081` and `--server.port=8082`), both pointing at the same Redis:

```

curl -X POST http://localhost:8081/counter/increment
curl -X POST http://localhost:8082/counter/increment
curl http://localhost:8081/counter/value # → 2

```

Both instances see and increment the **same counter**, and the **lock** ensures that if both hit `/increment` at the exact same millisecond, they don't race and clobber each other — only one holds the lock at a time, cluster-wide. A plain Java `AtomicLong` or a `synchronized` block wouldn't work here at all, since each JVM instance would have its own separate copy in memory.

Running Redis locally to test it

```
docker run -d -p 6379:6379 redis
```

That's the whole demo — want me to extend it to also show a distributed `RMap` or a blocking `RBlockingQueue` shared across the same two instances?